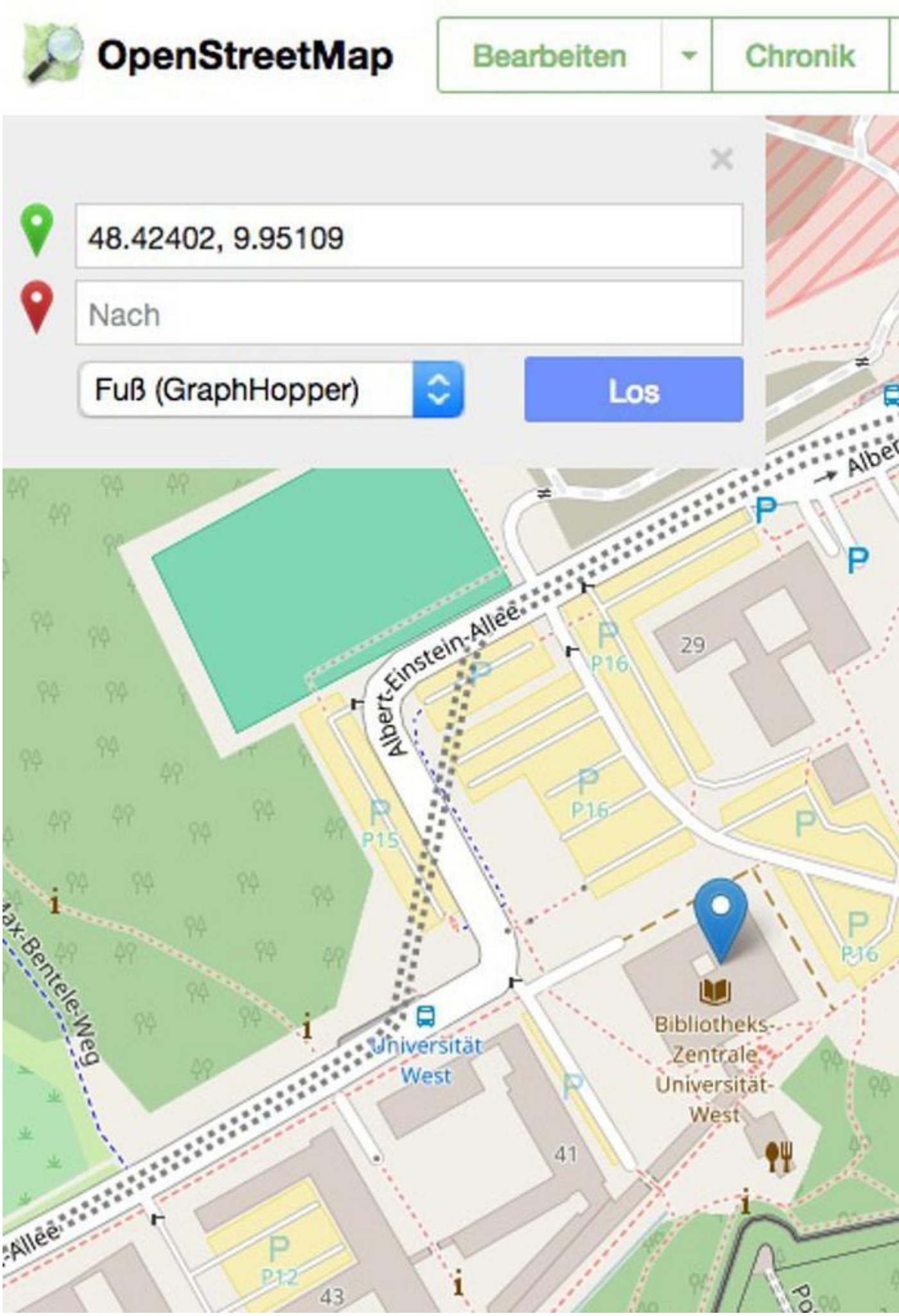
Final Implementation Plan: Campus Landmark A* Route Planner

This final plan combines your original PRD with all the requested advanced features: route restrictions, favourites, accessibility options, and step-by-step A visualization*. It also preserves the core requirements for OpenStreetMap data, custom A*, Dijkstra benchmarking, Folium maps, caching, and automated testing.

Campus_Landmark_AStar_Route_Planner_PRD.md

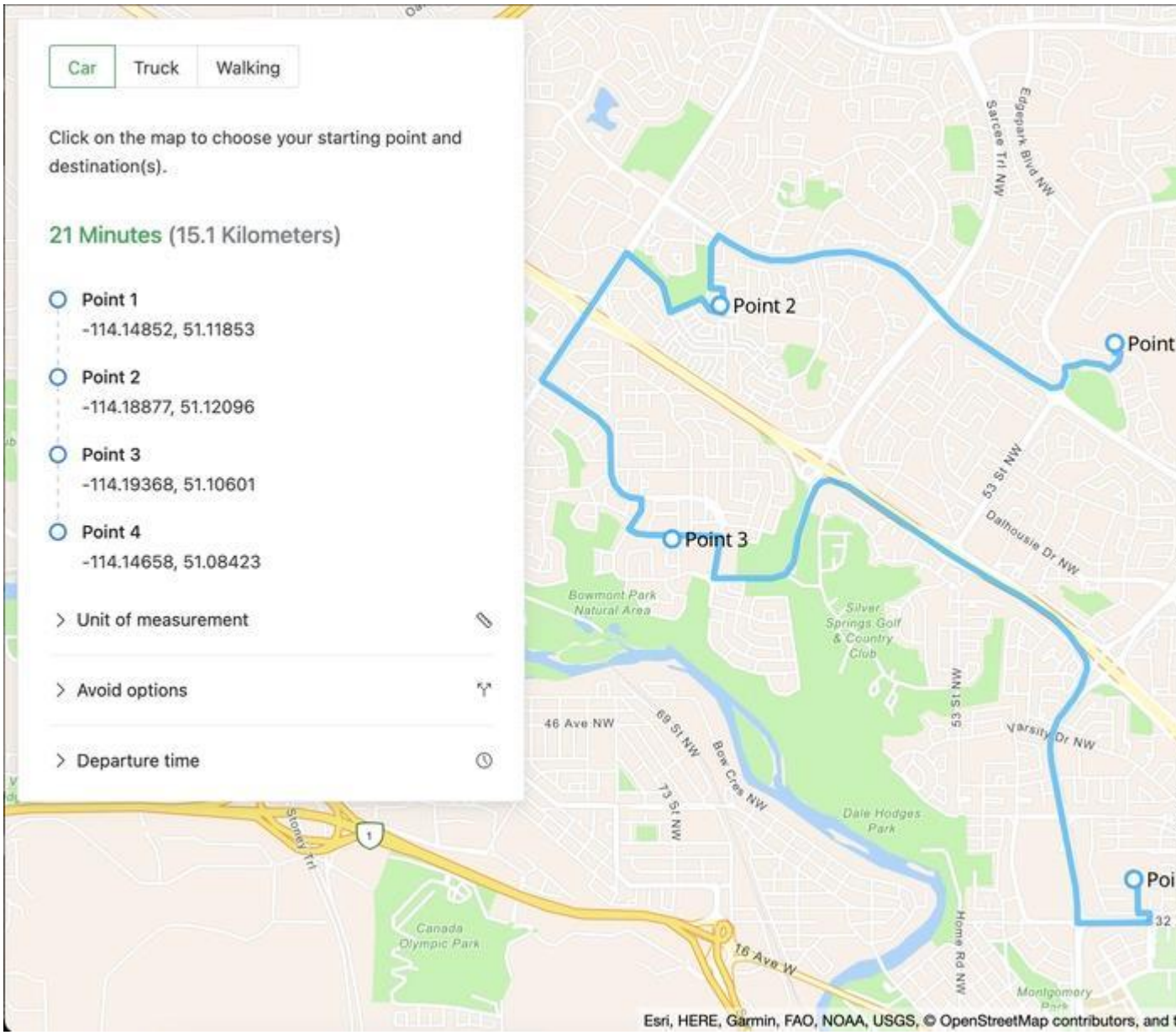
The implementation order is important: build and validate the routing engine first, then build the complete application around it.

1. Final project scope


OpenStreetMap
Bearbeiten
Chronik
48.42402, 9.95109
Nach
Fuß (GraphHopper)
Los
Albert-Einstein-Allee
P16
P15
29
Universität West
Bibliotheks-Zentrale Universität-West
41
43
P12

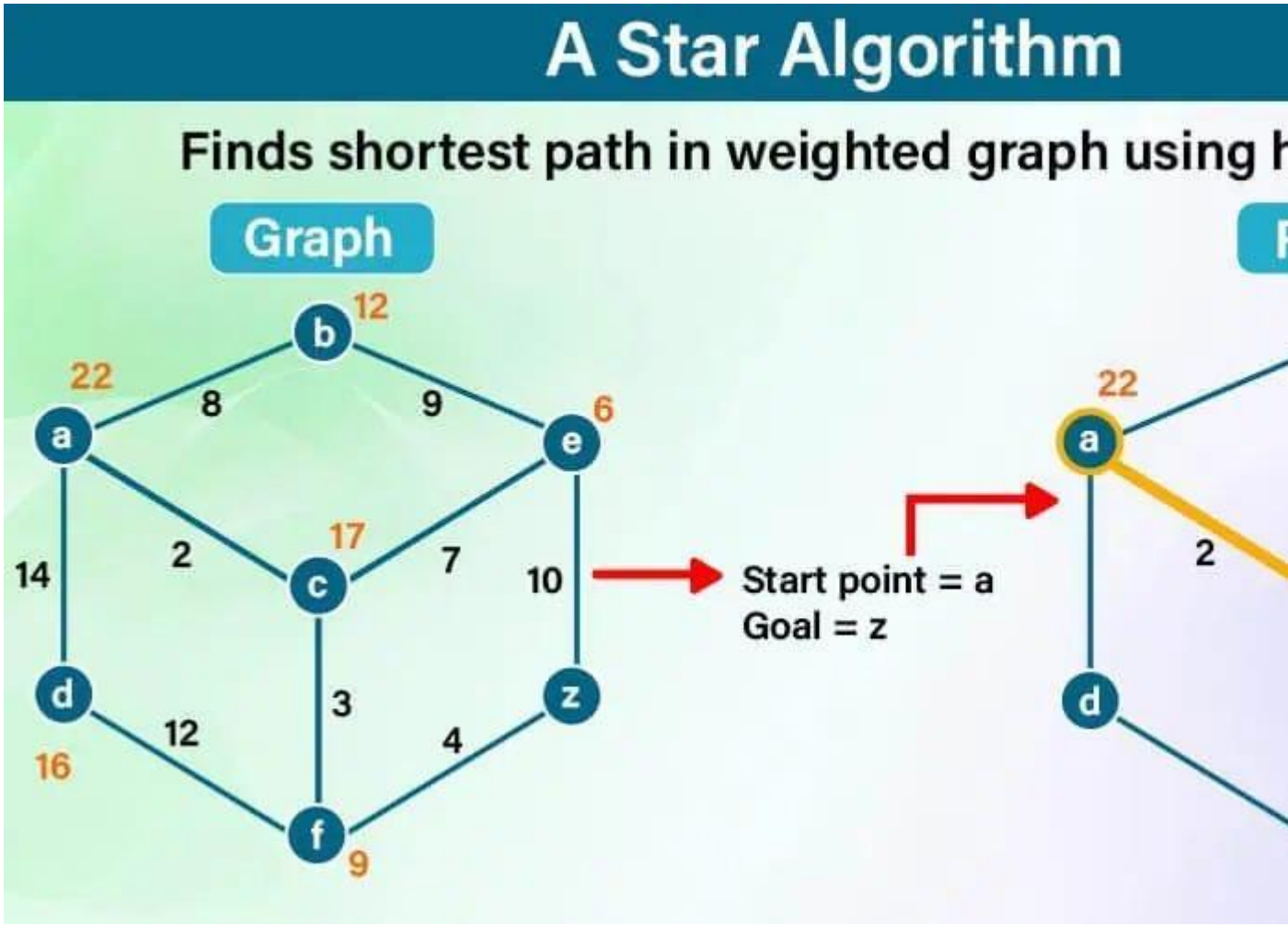
Core routing engine

OSMnx graph extraction, cached graph loading, landmark snapping, custom A*, custom Dijkstra, connectivity checks, and optimality verification.



Complete navigation application

Landmark search, route options, interactive map, route details, history, favourites, restrictions, and accessible-route preferences.



AI laboratory dashboard

A* exploration animation, g/h/f values, visited-node visualization, Dijkstra comparison, charts, and downloadable reports.

Recommended technology stack

Layer	Technology
Language	Python 3.10+
Road and pedestrian graphs	OSMnx, OpenStreetMap
Graph algorithms	NetworkX + custom `heapq` algorithms
Spatial indexing	SciPy / OSMnx
Application interface	Streamlit
Interactive map	Folium
Charts	Plotly or Matplotlib
Persistent user preferences	SQLite
Testing	pytest
Exports	JSON, CSV, HTML, optional PDF

For the first release, use a single Python application with Streamlit. You can later replace the interface with React and a Python API without rewriting the algorithm modules.

2. Final feature requirements

The following list should become the project's implementation checklist.

Feature implementation checklist

0/42

A. Core routing and map data

- [ ] Download OSM graphs for a selected campus or bounding box
- [ ] Support walking and driving networks
- [ ] Cache graphs locally using GraphML
- [ ] Convert MultiDiGraph to a directed graph using the minimum parallel-edge length
- [ ] Maintain a searchable landmark catalog
- [ ] Snap coordinates to graph nodes and warn beyond 250 metres
- [ ] Implement custom A* with Haversine heuristic
- [ ] Implement custom Dijkstra as the baseline
- [ ] Validate directed connectivity and handle unreachable destinations
- [ ] Render Folium maps with real edge geometry

B. Route restrictions

- [ ] Temporarily block a road or path segment
- [ ] Avoid selected edges or areas where the graph supports the restriction
- [ ] Allow stairs to be excluded when mapped attributes are available
- [ ] Recalculate the route after restrictions change
- [ ] Show why a route cannot be found
- [ ] Clear restrictions and restore the default route

C. Favourites and history

- [ ] Save a named favourite route
- [ ] Save favourite landmarks
- [ ] Quickly reuse a saved route
- [ ] Record recent searches and timestamps
- [ ] Rename and remove favourites
- [ ] Clear route history
- [ ] Persist preferences between application restarts

D. Accessibility

- [ ] Provide a wheelchair-friendly route preference
- [ ] Avoid stairs and steps when data identifies them
- [ ] Allow avoidance of known steep or inaccessible segments
- [ ] Display route warnings when accessibility information is incomplete
- [ ] Report missing ramps, broken paths, or incorrect map attributes
- [ ] Label accessibility results as estimated from available map data

E. Algorithm learning mode

- [ ] Show every meaningful A* expansion step
- [ ] Display g(n), h(n), and f(n) for candidate nodes
- [ ] Distinguish unexplored, frontier, expanded, and final-path nodes
- [ ] Show the current priority queue and predecessor links
- [ ] Provide Play, Pause, Next, Previous, and Reset controls
- [ ] Compare A* and Dijkstra on the same graph
- [ ] Show the final path, distance, expanded nodes, and elapsed time

F. Reporting and application quality

- [ ] Show route distance and estimated travel time
- [ ] Display algorithm metrics and comparison charts
- [ ] Export JSON and CSV reports
- [ ] Save interactive HTML maps
- [ ] Handle missing data, invalid input, and network failures
- [ ] Provide setup instructions and automated tests

Copy checklist

3. Final architecture

Separate the system into six independent layers so features can be implemented and tested without breaking the routing engine.

1. Streamlit user interface

Search · Navigation · Favourites · Accessibility · Learning mode

2. Application service layer

Validate requests · Manage user preferences · Apply restrictions · Coordinate routing

3. Graph manager

OSMnx · Cache · Preprocessing · Snapping

4. Routing engine

A* · Dijkstra · Heuristic · Connectivity

5. Validation and event recording

Path verification · Search trace · Performance metrics · Error reporting

6a. Map and reports

Folium · Charts · JSON · CSV

6b. Persistence

GraphML · SQLite · Export files

Design rule: restrictions and accessibility preferences must change which edges are eligible for a route. They must not be implemented as visual overlays alone.

4. Final project directory structure

```
campus-landmark-astar/
│
├── main.py
├── requirements.txt
├── README.md
├── .gitignore
├── pyproject.toml
│
├── src/
│   ├── config.py
│   ├── exceptions.py
│   ├── landmarks.py
│   │
│   ├── graph/
│   │   ├── graph_manager.py
│   │   ├── preprocessing.py
│   │   ├── snapping.py
│   │   └── connectivity.py
│   │
│   ├── algorithms/
│   │   ├── astar.py
│   │   ├── dijkstra.py
│   │   ├── heuristics.py
│   │   ├── search_trace.py
│   │   └── metrics.py
│   │
│   ├── routing/
│   │   ├── route_service.py
│   │   ├── restrictions.py
│   │   ├── accessibility.py
│   │   ├── route_validator.py
│   │   └── directions.py
```

```
│   │
│   ├── storage/
│   │   ├── database.py
│   │   ├── favourites.py
│   │   └── history.py
│   │
│   ├── visualization/
│   │   ├── map_view.py
│   │   ├── benchmark_charts.py
│   │   └── learning_view.py
│   │
│   ├── exports/
│   │   └── report_exporter.py
│   │
│   └── ui/
│       ├── dashboard.py
│       ├── navigation_page.py
│       ├── favourites_page.py
│       ├── analytics_page.py
│       └── learning_page.py
│
├── data/
│   ├── landmarks.json
│   └── graphs/
│
├── tests/
│   ├── test_heuristics.py
│   ├── test_astar.py
│   ├── test_dijkstra.py
│   ├── test_graph_pipeline.py
│   ├── test_connectivity.py
│   ├── test_restrictions.py
│   ├── test_accessibility.py
│   ├── test_favourites.py
│   ├── test_search_trace.py
│   └── test_integration.py
│
└── reports/
```

Generated graph files and reports should be excluded from Git by default, unless you deliberately include a small, licensed test fixture.

5. Detailed implementation sequence

Phase 1 — Environment and configuration

Implement: `config.py`, `exceptions.py`, `requirements.txt`, `main.py`.

Tasks:

1. Create and activate a Python virtual environment.
2. Install OSMnx, NetworkX, SciPy, Folium, Streamlit, Matplotlib, pytest, and any other required dependencies.
3. Configure data directories and the default campus area.
4. Define shared units, logging rules, error messages, and settings.
5. Create a small synthetic graph for offline algorithm tests.

Deliverable: an application skeleton that starts reliably.

Phase 2 — Graph acquisition and preprocessing

Implement: `graph_manager.py`, `preprocessing.py`, `snapping.py`, `connectivity.py`.

Tasks:

1. Download the required OpenStreetMap graph.
2. Support walking and driving network modes.
3. Cache the raw graph to GraphML.
4. Reload a compatible cache when available.
5. Validate edge attributes and distance units.
6. Convert the `MultiDiGraph` to a directed `DiGraph`, retaining the minimum-length parallel edge for each directed pair.
7. Preserve usable edge attributes and geometry.
8. Build coordinate snapping and the 250 m warning.
9. Add weak-connectivity checks for early rejection.
10. Retain directed reachability validation for one-way paths.

Critical graph rule: never automatically add reverse edges to make the graph connected. A one-way path must remain one-way.

Deliverable: a stable, reusable directed graph.

Phase 3 — Custom A* and Dijkstra

Implement: `astar.py`, `dijkstra.py`, `heuristics.py`, `metrics.py`.

Tasks:

1. Implement the Haversine distance in meters.
2. Implement A* using `heapq` and \(f(n)=g(n)+h(n)\).
3. Implement Dijkstra independently using \(h(n)=0\).
4. Reconstruct paths with a predecessor map.
5. Support identity routes, invalid nodes, and unreachable destinations.
6. Record expanded nodes, peak frontier size, execution time, and total distance.
7. Validate edge weights and priority-queue handling.
8. Verify the heuristic's admissibility and consistency on suitable test graphs.

The Haversine heuristic is admissible for shortest-distance routing only when the graph's edge costs are compatible with geographic distance. If you later optimize for time, stairs avoidance penalties, or other costs, the heuristic must be reconsidered to ensure it remains a lower bound on that new cost.

Deliverable: two tested algorithms that agree on optimal path costs.

Phase 4 — Route validation and baseline dashboard

Implement: `route_service.py`, `route_validator.py`, `dashboard.py`, `navigation_page.py`.

Tasks:

1. Add landmark selection and custom coordinate input.
2. Validate the start and destination.
3. Run A* on the processed graph.
4. Validate each edge in the returned route.
5. Recalculate the path length independently.
6. Run Dijkstra for benchmark comparisons.
7. Render the route using Folium.
8. Show distance, estimated time, route status, and performance metrics.

Do not build the advanced features until the baseline application passes its route tests.

Deliverable: an end-to-end application with correct routes.

Phase 5 — Route restrictions

Implement: `restrictions.py`.

Support three categories of restrictions:

- Edge restrictions: exclude selected graph edges or road segments.
- Area restrictions: avoid graph edges intersecting a defined blocked area.
- Attribute restrictions: avoid edges marked as stairs, inaccessible, or otherwise disallowed.

Implementation requirements:

1. Assign a stable internal identifier to each directed edge, such as `(u, v)`.
2. Store active restrictions separately from the base graph.
3. Build a route-specific eligible graph view or equivalent edge filter.
4. Apply the same eligible-edge rules to A* and Dijkstra.
5. Recalculate when restrictions change.
6. Explain when the restrictions make the destination unreachable.
7. Allow the user to clear restrictions.

Important: do not destructively remove edges from the cached base graph. Restrictions should be reversible and should not affect other route requests.

If a multi-edge road segment is represented by several graph edges, the restriction manager should identify and exclude the correct segments, not accidentally block unrelated parallel roads.

Deliverable: a route planner that respects user-selected detours and closures.

Phase 6 — Accessibility options

Implement: `accessibility.py`, with updates to the graph pipeline and route service.

Recommended options:

- Avoid stairs and steps.
- Prefer mapped wheelchair-accessible paths.
- Avoid known inaccessible or restricted segments.
- Display warnings where slope, surface, width, curb ramps, or step-free access data is missing.
- Allow users to submit a map-data correction.

For the first version, implement hard exclusions for clearly identified disallowed edges. Do not simply assign a huge penalty and claim the route is wheelchair-accessible.

OSM data may be incomplete or incorrect. The application should say “accessibility not verified” when it cannot confirm a route's relevant properties. It should not promise that a route is fully accessible based only on missing or partial map attributes.

Deliverable: an accessibility-aware routing mode with transparent limitations.

Phase 7 — Favourites and history

Implement: `database.py`, `favourites.py`, `history.py`.

Use SQLite to store local preferences and route metadata.

Suggested database tables:

Table	Important fields
`landmark_favourites`	ID, landmark ID, label, created timestamp
`route_favourites`	ID, route name, start ID, destination ID, options JSON
`route_history`	ID, start, destination, timestamp, distance, network mode
`saved_restrictions`	ID, name, restriction configuration

Tasks:

1. Create the SQLite schema.
2. Save and remove favourite landmarks and routes.
3. Restore saved route options.
4. Store recent route summaries.
5. Allow renaming favourites.
6. Provide a history-clear operation.
7. Handle stale landmark IDs and incompatible graph data safely.

Avoid storing unnecessary personal information. A local, single-user database is sufficient for the initial application.

Deliverable: persistent favourites and route history across restarts.

Phase 8 — Step-by-step algorithm visualization

Implement: `search_trace.py`, `learning_view.py`, `benchmark_charts.py`.

This is the main educational feature of the project.

For every meaningful A* search event, record a snapshot containing:

```
{
  "step": 12,
  "current_node": "node_42",
  "frontier": ["node_17", "node_29"],
  "expanded_nodes": ["node_01", "node_08", "node_42"],
  "g_score": 250.0,
  "h_score": 180.0,
  "f_score": 430.0,
  "event": "expand"
}
```

This is an illustrative trace schema; actual node IDs and scores must come from the search engine.

Visualization controls

- Play and pause.
- Next and previous step.
- Reset.
- Adjustable playback speed.
- Display the current node and its predecessor.
- Show the frontier and expanded nodes.
- Show \(g(n)\), \(h(n)\), and \(f(n)\).
- Display the final shortest path.
- Switch between A* and Dijkstra.
- Compare both algorithms' expansion counts and execution times.

Store trace events only when learning mode is enabled. This prevents unnecessary memory use during normal routing.

For repeatable step-by-step comparison, use a deterministic tie-breaking rule and the same graph, start, destination, and edge costs.

Deliverable: an interactive explanation of how informed search explores the campus network.

Phase 9 — Reports and exports

Implement: `report_exporter.py`.

Export:

- JSON route details.
- CSV benchmark results.

- Folium interactive HTML maps.
- Optional PDF reports with the route summary, metrics, and screenshots.

Include the algorithm name, graph/network mode, start and destination, path distance, search metrics, active restrictions, accessibility settings, and any warnings.

A report must distinguish measured execution time from estimated travel time.

Phase 10 — Final integration and release

Tasks:

1. Run all automated tests.
2. Test route restrictions with one-way edges.
3. Test accessibility settings against known tagged and untagged edges.
4. Verify favourite persistence.
5. Test every learning-mode control.
6. Check invalid inputs and network failures.
7. Verify report exports.
8. Measure performance with a preloaded graph.
9. Write the README and setup instructions.
10. Prepare screenshots, a sample report, and an AI laboratory demonstration.

Deliverable: a reproducible, tested project ready for demonstration and submission.

6. Complete 14-day development schedule

Day	Main task	Completion milestone
1	Environment and project structure	Application starts
2	OSMnx graph download and caching	Graph loads from cache
3	Preprocessing, snapping, connectivity	Directed graph validated
4	Haversine and A*	A* passes core tests
5	Dijkstra and cost verification	Both algorithms agree
6	Route validation and metrics	Baseline benchmarks work
7	Folium map and landmark search	Basic navigation works
8	Streamlit dashboard	End-to-end UI works
9	Route restrictions	Blocked paths are avoided
10	Accessibility options	Supported restrictions are enforced
11	Favourites and history	Preferences persist
12	Algorithm learning mode	Step controls and trace work
13	Exports, integration, regression tests	Features work together
14	Documentation and demonstration	Submission package is ready

This schedule assumes the developer already has basic Python knowledge and can work consistently. Campus data issues or extensive UI refinement may require additional time.

7. Final acceptance criteria

The project is ready for release when all critical requirements below pass.

Algorithm correctness

A* and Dijkstra return equal optimal path costs on every tested reachable pair. Invalid and unreachable requests are handled safely.

Graph integrity

One-way restrictions are preserved, snapping warnings work, and restrictions never corrupt the cached base graph.

Navigation and accessibility

Valid routes are drawn on the map, user restrictions are enforced, and incomplete accessibility data produces explicit warnings.

Persistence

Favourites and history survive application restarts, and the user can edit or clear them.

Educational visualization

The learning mode shows search states in a reproducible sequence and accurately reflects the algorithm's actual execution.

Performance and documentation

The benchmark results are measured, the tests are reproducible, and the application can be installed using the README instructions.

8. Final performance targets

Use the targets from your original PRD, but measure and report them rather than assuming they will be achieved.

Campus_Landmark_AStar_Route_Planner_PRD.md

Metric	Target
Optimal path cost	100% agreement between A* and Dijkstra on tested reachable pairs
Search efficiency	At least 40% fewer expanded nodes on the defined qualifying benchmark set
Search latency	Up to 500 ms on a preloaded graph with 50,000 edges, on the specified test hardware
End-to-end processing	Under 3 seconds for the specified area and conditions, excluding or explicitly reporting external network latency

Disconnected-route handling	Fast rejection when cached connectivity data proves no route can exist
Test coverage	All critical route, restriction, persistence and visualization tests pass

The performance results should include the hardware, graph size, cache status, route sample, and measurement method. If the 40% target is not met, report the actual results and investigate the graph topology and heuristic rather than manipulating the metrics.

9. Final deliverables

At completion, the project should contain:

1. Complete Python source code.
2. A custom A* algorithm and independent Dijkstra implementation.
3. A reusable graph download, caching, and preprocessing pipeline.
4. A campus landmark catalog.
5. An interactive Streamlit dashboard with a Folium map.
6. Route restrictions and accessibility options.
7. Favourites and route history backed by SQLite.
8. Step-by-step algorithm visualization.
9. Automated unit and integration tests.
10. Benchmark charts and reports.
11. CSV, JSON, and HTML export functionality.
12. A complete README with installation, execution, testing, limitations, and screenshots.
13. A demonstration scenario showing the same route under normal, restricted, and accessibility-aware conditions.

10. Final implementation order

Follow this exact dependency order:

01

Build the graph pipeline

02

Implement and test A*

03

Implement Dijkstra and verify optimality

04

Add route validation and benchmarking

05

Build the map and basic dashboard

06

Implement route restrictions

07

Add accessibility-aware routing

08

Implement favourites and history

09

Add step-by-step algorithm visualization

10

Integrate exports, testing, and documentation

My final recommendation: build this as a Python + Streamlit application first, with a modular routing engine and SQLite persistence. That provides a manageable implementation while demonstrating algorithms, graph theory, geospatial processing, software testing, database design, UI development, and data visualization in one project.

The key is that every advanced feature must use the same validated routing engine. That way, a favourite route, a blocked path, an accessibility preference, and an A* learning session all rely on one consistent source of routing logic rather than separate implementations.